



Nonce as a Lock: Same-Slot Proposer Races with Frame Transactions



TL;DR: Hegotá gives based rollups no native exclusive right to propose. What EIP-8141 frame transactions can do is have every proposal use the same gate contract `G` as its sender and share `G`'s nonce. The loser of a same-slot race then goes from "lands, reverts, and still pays execution gas plus a non-refundable blob fee" to "invalid at the consensus layer, includable in no valid block, pays nothing".

1. Design

Every proposal uses a non-upgradeable gate contract `G` as `tx.sender`, and the proposer `P` pays.

- **The nonce is the lock:** before executing any frame, EIP-8141 checks `tx.nonce == nonce(G)`, and it increments `G`'s nonce at payment approval. At most one transaction sharing `(G, n)` can be included; the losers' nonce no longer matches, so they are invalid and pay neither execution gas nor blob fees.
- **Expiry is the TTL:** the first frame is `EXPIRY_VERIFIER (0x8141)`, which requires `block.timestamp <= expiry`, so the transaction becomes invalid once the target slot has passed. It gives only an upper bound; there is no not-before.
- **G only checks shape:** in a `VERIFY` frame it uses introspection to pin down the whole transaction, then calls `APPROVE (APPROVE_EXECUTION)`. It reads only the transaction itself, with no storage reads and no banned opcodes, so the transaction is valid for the public mempool.
- **The Inbox stays plain Solidity:** it only adds a `proposeFor` that trusts `G` alone; the proposer's identity comes from calldata `G` has validated. No new opcodes are needed.
- **It fixes who pays, not who wins:** the L1 builder still picks the winner by tip, L2 value still flows to L1 as tips, and there is no advance exclusivity. Exclusivity is per nonce, not per block (§3).

1.1 Transaction structure

Envelope: `[chain_id, nonce = nonce(G), sender = G, frames, signatures, fees, blob_versioned_hashes]`. Every frame has value 0.

#	mode	flags	target	limits [exec, state]	data	mempool role
0	VERIFY	0	<code>0x...8141</code>	<code>[3_200, 0]</code>	<code>uint64be(T_S)</code>	expiry_verify
1	VERIFY	2	null (i.e. G)	<code>[5_000, 0]</code>	empty	only_verify
2	VERIFY	1	P (proposer EOA)	<code>[3_500, 0]</code>	empty	pay (default code)
3	SENDER	0	Inbox	<code>[≥MIN_EXEC, from simulation]</code>	<code>proposeFor(P, packedInput)</code> , 68 bytes	user_op

Signatures: `[[[ARBITRARY, "", "", ""], [SECP256K1, P, "", sig_P]]]`. Hard constraints from the spec:

- **G comes before P:** `APPROVE_PAYMENT` requires `sender_approved == true`, and that is exactly the step that increments the nonce.
- **sig0 is a zero-length placeholder:** in a frame that only approves payment, the EOA default code reads the signature at index 1, so index 0 must have an entry. Its length is pinned to 0; otherwise a relayer could pad it to inflate P's fee and change the tx hash.
- **sig1 names P as signer and leaves msg empty:** an empty signer resolves to `tx.sender` (i.e. G), which fails static signature validation. With `msg` empty, P signs `compute_sig_hash(tx)`, which covers the whole frame list, so P's signature binds the entire transaction.
- **Gas budget:** the cold access to a frame's target is charged to that frame's own `limits.execution`, and `0x8141` is not pre-warmed. EIP-8038 raises cold access to 3,000, so frame 0 gets 3,200 and frame 2 gets 3,500, and P must be an EOA with no 7702 delegation (otherwise the delegate code runs and goes OOG). The VERIFY part totals about 14k gas, far below `MAX_VERIFY_GAS = 100k`.
- **Timestamps:** the upper bound on L2 block timestamps is the timestamp of the L1 block the proposal actually lands in, not `T_S`. Out-of-range values are not clamped: the entire derivation source is replaced with a default manifest containing only the anchor. Since EXPIRY has no not-before, a transaction signed before the slot S-1 block is seen can land in S-1, so the upper bound is the timestamp of the earliest slot it could land in (equal to `T_S` only when signing after seeing the slot S-1 block). The lower bound reserves 1 second for each forced-inclusion block ordered ahead.

1.2 ProposalGate (G)

About 120–150 bytes (estimated), non-upgradeable, no storage, and no CALL / DELEGATECALL / CREATE of any kind. solc inline assembly does not recognize `0xaa` or `0xb0-0xb5`, so G has to be written in standalone Yul with `verbatim_*`, in Huff, or as hand-written bytecode. Foundry and revm cannot run it; it can only be tested on EEST, hive, or a frames devnet.

```
// immutable: INBOX, SEL = selector("proposeFor(address,bytes32)"), MIN_EXEC, MIN_ST
// ATE, MIN_TIP
fallback() { // in a non-frame tx the first TXPARAM halts exceptionally, so plain
calls always fail
    require(txparam(0x02) == address(this)); // sender == G
    require(txparam(0x09) == 4); // exactly 4 frames: approval authori
zes every later SENDER frame
```

```

    require(txparam(0x0A) == 1);          // this is frame 1
    require(txparam(0x0B) == 2);          // exactly 2 signatures
    require(txparam(0x03) >= MIN_TIP);    // max_priority_fee floor: every pool
able version is a real bid; max_fee is uncapped
    uint256 nBlobs = txparam(0x07);
    require(nBlobs >= 1);

    require(frameparam(0x02, 0) == VERIFY && frameparam(0x00, 0) == 0x8141); // exp
    ired: check only mode and target, not data
    require(frameparam(0x02, 1) == VERIFY && frameparam(0x03, 1) == 2);      // sel
    f
    require(frameparam(0x02, 2) == VERIFY && frameparam(0x03, 2) == 1);      // pay
    uint256 P = frameparam(0x00, 2);

    require(sigparam(0x01, 0) == ARBITRARY && sigparam(0x03, 0) == 0); // placehold
    er length pinned to 0: no fee or tx-hash malleability
    require(sigparam(0x01, 1) == SECP256K1 && sigparam(0x02, 1) == 0 && sigparam(0x0
    0, 1) == P);

    // frame 3: the only SENDER frame; static ABI, calldata is always 68 bytes
    require(frameparam(0x02, 3) == SENDER && frameparam(0x00, 3) == INBOX);
    require(frameparam(0x03, 3) == 0 && frameparam(0x08, 3) == 0); // no scope, no
atomic batch, value 0
    require(frameparam(0x01, 3) >= MIN_EXEC && frameparam(0x09, 3) >= MIN_STATE);
    require(frameparam(0x04, 3) == 68 && framedataload(3, 0) >> 224 == SEL);
    require(framedataload(3, 4) == P); // the _proposer argument m
ust be P

    uint256 w = framedataload(3, 36); // packed ProposeInput in L
ibCodec's big-endian layout
    require((w >> 208) == 0); // deadline == 0: the time
upper bound comes only from the expiry frame
    uint256 start = (w >> 192) & 0xffff;
    uint256 n = (w >> 176) & 0xffff;
    require(n >= 1 && start + n <= nBlobs); // blob references must fal
l within this tx's blobs
    require(((w >> 136) & 0xffff) == 0); // numForcedInclusions ==
0: the Inbox processes due FIs automatically

    approve(2); // APPROVE_EXECUTION
}

```

- **Load-bearing pins:** a shared nonce turns "lands and reverts" from one proposer's loss into a wasted slot, so any revert that someone else can trigger on the propose path is a lever for burning the nonce. Safety actually comes from: exactly 4 frames with their modes and flags (execution approval authorizes every later SENDER frame); the target and `value == 0` of the single SENDER frame; `_proposer` = pay-frame target = sig1 signer; a zero-length placeholder signature; and `deadline` and `numForcedInclusions` pinned to 0 (a past deadline or an under-reported FI count would let the winner revert at will). `MIN_EXEC`, the blob count, and the

blob references constrain only the attacker's own transaction; they are mainly guardrails against a client setting gas too low and going OOG itself.

- **Static ABI:** the deployed `LibCodec` accepts non-canonical offsets, dirty padding, and `_data` of any length, so a dynamic `bytes` layout would need 6 load-bearing pins. `proposeFor(address,bytes32)` removes all of them, along with the whole class of bugs where G and the Inbox decode different results. With `expectedProposalId` added, the 21-byte `ProposeInput` still fits in one word.
- **Stack-order trap:** `FRAMEPARAM` / `SIGPARAM` take index on top of the stack and param second; `FRAMEDATALOAD` takes offset on top and frameIndex second; `APPROVE` takes offset, length, scope from the top down. Yul `verbatim` puts the leftmost argument on top, so copying the pseudocode's reading order silently reads the wrong field instead of halting. Helpers consistently take `(index, param)` and `(offset, frameIndex)`, and are differential-tested against EELS.
- **Why not make the Inbox G:** the Inbox holds bonds and forced-inclusion fees, so one missing constraint in VERIFY would let anyone make arbitrary calls with `msg.sender = Inbox`. A minimal G holds no assets, so its blast radius is close to zero. The only cost is that G cannot read Inbox storage, which this design doesn't need.

Invariants the audit must hold:

1. The Inbox sees `msg.sender == G` if and only if a frame tx's VERIFY ran G's code and every check passed (only code with `ADDRESS == tx.sender` can approve execution; G has no CALL / DELEGATECALL; other transaction types cannot have a contract as sender; 7702, DELEGATECALL, and CREATE2 cannot forge it).
2. The single SENDER frame calls `proposeFor`, and `_proposer` is both the payer and the signer.
3. G is non-upgradeable and is the Inbox's root of trust. The Inbox never reads `tx.origin` (ORIGIN is G inside the SENDER frame). G and the Inbox are versioned as a pair per fork, and G is not deployed on any persistent network before the spec is frozen.

1.3 Inbox changes

```
error NotProposalGate();
error DirectProposeDisabled();
address internal immutable _proposalGate;    // 0 = disabled
bool internal _directProposeDisabled;        // governance switch: propose has no whe
nNotPaused

function propose(bytes calldata _lookahead, bytes calldata _data) external nonReentr
ant {
    require(!_directProposeDisabled, DirectProposeDisabled()); // disabled once G h
as soaked, re-enabled if G breaks
    _propose(msg.sender, _lookahead, LibCodec.decodeProposeInput(_data));
}

function proposeFor(address _proposer, bytes32 _packedInput) external nonReentrant {
    require(_proposalGate != address(0) && msg.sender == _proposalGate, NotProposalG
ate());
    _propose(_proposer, "", LibCodec.decodeProposeInput(_packedInput));
}
```

`_buildProposal` takes a new `_proposer` parameter that replaces the 4 uses of `msg.sender` on the propose path: `Inbox.sol:596` (forced-inclusion fee recipient), `:603` (`checkProposer`), `:606` (`hasSufficientBond`), and `:615` (the `proposer` field). The `Proposal` struct and the `Proposed` event are unchanged, so derivation, the prover, and the eventindexer need no changes.

Required in the same upgrade:

- **Process forced inclusions by due time:** `toProcess = min(max(requested, due), available, 10)`, so due entries are processed automatically and G pins `numForcedInclusions` to 0. The current rule consumes `min(requested, available, 10)` whether or not entries are due; combined with `0xffff`, it lets anyone stuff FIs into someone else's proposal in the same block. One FI pushes gas from 76,485 to 100,517, ten push it to 218,805, and each one also shifts P's timestamps 1 second later.
- **Pull-based forced-inclusion fees:** `sendEtherAndVerify` forwards all remaining gas, so a P that rejects ETH or burns the gas reverts and burns the nonce, and G cannot tell in VERIFY whether P has a 7702 delegation. Switch to accounting plus a `nonReentrant claimFees`: append a `mapping(address => uint256)` at slot 258, shrink `__gap` from 43 to 42, use reinitializer 4, and regenerate both the Mainnet and Devnet layout files. Sweeps must exclude queued fees and unclaimed credits.
- **expectedProposalId:** `ProposeInput` grows from 15 to 21 bytes. A reverting SENDER frame consumes the nonce without producing a proposal, and L1 reorgs and the direct path also decouple the nonce from the proposal id, so the field is not redundant; before G is live it is also the main defense against cross-block stale submissions. An `expectedParentProposalHash` can be added to guard against reorgs that keep the id but replace the parent's content. Clients never derive the proposal id from the nonce.
- **Open the checker:** `_proposerChecker` is immutable, but it points to an EIP-1967 proxy, so upgrading the checker to allow anyone is enough.
- **No bond threshold on the G path:** mainnet Shasta's `minBond` has been 0 since activation and bonds are optimistic; spam is bounded by one proposal per block (PR #20186). An extra bond check would only add another revert, letting an under-bonded winner use it to burn the nonce and skip FIs. If a threshold is really needed, mirror `bond0k[P]` into G.

Deployment order: deploy G with CREATE2 (with the Inbox proxy address as an immutable), then deploy a new Inbox implementation that knows G's address, then upgrade through the DAO.

1.4 How a race plays out

1. **Slot S-1:** after seeing the slot S-1 block, A and B both read `nonce(G) = n`, build on the same L2 head (heavy transaction overlap is fine), and each sign a transaction with `expiry = T_S`.
2. **Propagation:** both can enter the public mempool and both can go directly to builders. Replacing a `(G, n)` transaction requires raising both 1559 fee fields by at least 10%; blob-carrying variants need all three fee caps raised by 100% in the reth and geth blobpools.
3. **Slot S:** the builder picks the `(G, n)` variant with the highest total tip among those it has seen, say A's. Once A lands, G's nonce becomes `n+1` at the pay frame.
4. **What happens to B:** its nonce no longer matches, so no valid block can include it; nodes drop it on revalidation, and B pays nothing. A builder that forces it in produces an EL-invalid payload while its bid is still paid, so this does not rely on builder good behavior.
5. **Slot S+1:** B reads `nonce(G) = n+1`, drops the transactions A already included, and rebuilds with `expiry = T_{S+1}`.

6. **If nobody lands in slot S:** every variant becomes invalid once `T.S` passes (nodes use a strict `<`, so they drop them one slot late), everyone competes for the same `n` again in `S+1`, and nobody pays.

2. Cost

	Today (type-3 tx)	This design
Same-block loser	30,944 gas + full blob fee	0
Winner	75,594 gas + blob fee	About 13.5k gas more than a type-3 tx on the same fork, roughly +18% (estimated)

- The overhead comes mostly from the fixed frame cost, signature verification, cold accesses to `0x8141` and `P`, and signature bytes, estimated with the post-Glamsterdam gas schedule (EIP-2780, EIP-8038). The 75,594 baseline needs to be re-simulated on Glamsterdam or a frames devnet.
- The value is structural, not savings:** it only breaks even if each proposal averages about 0.4–0.5 losers that would otherwise have landed on chain; during the 2025 guard era the figure was about 0.002 (estimated). Today's application-level guard plus private, revert-protected builder submission is only builder policy: violations carry no protocol penalty, and copies that leak into the public mempool still land and pay. This design does not depend on builder policy, costs nothing even for participants who use only the public mempool, and removes the tail risk of blob fees (in the congested windows of 2024-11 and 2025-02, blob fees were 34% and 38% of total proposer spend).

3. Limitations

- Exclusivity is per nonce, not per block:** if someone lands `n+1` right after `n` in the same block, `n+1` reverts at the Inbox's `CannotProposeInCurrentBlock` and pays, which hurts only whoever pre-signed `n+1`. The spec's "one pending transaction per sender" is only a SHOULD, and neither reth nor Nethermind implements it, so clients do not pre-sign `n+1` before `n` is confirmed, and the Inbox keeps its same-block check.
- Single-seat squatting:** `G` cannot read `TIMESTAMP`, so it cannot bound how far in the future the expiry is. A squatter can sign a version with a distant expiry, zero tip, and a very high `max_fee`; a replacement then has to raise the 1559 fields by 10% (100% with blobs) and hold enough balance, and under pool pressure nodes evict the transactions closest to expiry first. The countermeasure is `G`'s `MIN_TIP` pin (with no cap on `max_fee`), which makes every poolable version a real bid that a builder would include. Once EIPs PR #12252 is merged, `G v2` can also pin `expiry - valid_after ≤ 12-24 s`. Serious proposers also submit directly to builders.
- Reverts that can still burn the nonce:** after the §1.3 changes and `G`'s pins on `deadline` and `numForcedInclusions`, the only reverts others can still trigger are a full ring buffer (which requires provers to stall and hits every proposer equally) and `n+1` in the same block (which hurts only the pre-signer). Censorship cost is set by the tip auction: to get any `(G, n)` included, an attacker first has to outbid honest proposers at the builder.
- `G` is tied to a Draft spec:** `G` hard-codes opcode numbers, the shape of the expiry frame, and the sig hash layout. EIPs PR #12198 (expiry becomes a new mode), #12252 (data becomes a 16-byte `valid_after|expiry`), or the activation of EIP-8250 would each make `G` fail closed, and the main path would stay down until `G v2` plus an Inbox upgrade get through the DAO. Hence `G` is versioned per fork and reads only the expiry frame's mode and target.
- Direct propose keeps a governance switch:** it is disabled once `G` has soaked and re-enabled if `G` breaks. Until then, the Inbox's same-block revert still arbitrates between the two paths, and the zero-cost

guarantee holds only within the G path.

4. References

- [Full research report](#) (on-chain sampling and a comparison of alternatives)
- [Review report](#) (2026-09-23)
- [EIP-8141 Frame Transaction](#) (Draft, [b75cbe6115](#)), [EIP-7732 ePBS](#), [EIP-8038](#), [EIP-2780](#)
- EIPs PRs: [#12252](#) ([valid_after](#)), [#12198](#) (expiry as a new mode)
- Taiko: [PR #18570](#) (conditional propose), [PR #20186](#) (one proposal per block), [PR #19488](#) (lessons from parentMetaHash)